Introduction

Number System

	Base	
Decimal	10	0,1,2,3,4,5,6,7,8,9
Binary	2	0,1
Hexadecimal	16	0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
Octal	8	0,1,2,3,4,5,6,7

Number System

232	2	0
116	2	0
58	2	0
29	2	1
14	2	0
7	2	1
3	2	1
1	2	1
0		



$232_{10} = 11101000_2$

11101000_2

$232_{10} = E8_{16}$

11101000_2

$232_{10} = 350_8$

Number System

	0.3	x 2	
	0.6	x 2	0
0.2	1.2	x 2	1
	0.4	x 2	0
	0.8	x 2	0
0.6	1.6	x 2	1
	1.2		



$0.3_{10} = 0.010011001100\ldots_2 = 0.\overline{01001}_2$

$0.010011001100\ldots_2$

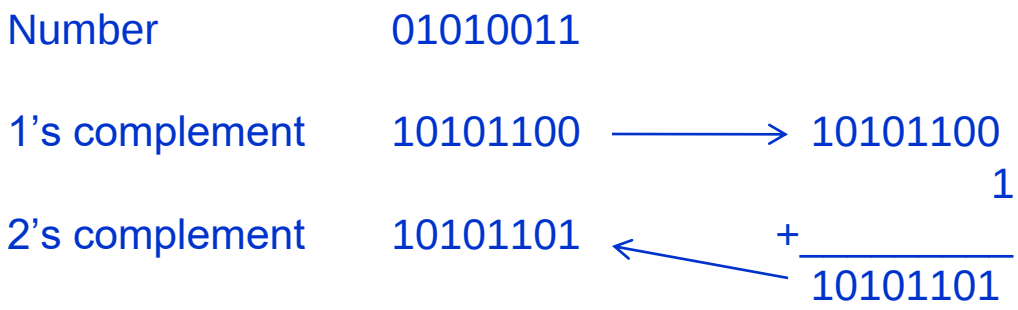
$0.3_{10} = 0.4CC\ldots_{16} = 0.4\overline{C}_{16}$

$0.010011001100\ldots_2$

$0.3_{10} = 0.2314\ldots_8$

Number System

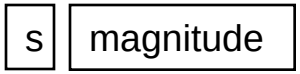
Complements



Decimal	Binary
-8	1000
-7	1001
-6	1010
-5	1011
-4	1100
-3	1101
-2	1110
-1	1111
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Integer Representation

Sign-magnitude representation



+5 0 00000101

-5 1 00000101

2's complement representation

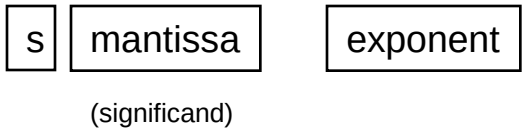
+5 00000101

-5 11111011

Floating-point Representation

mantissa x radix^{exponent}

radix=2



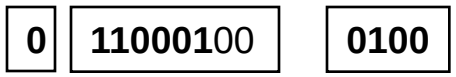
Sign-magnitude representation

2's complement representation

Ex: +12.25

1100.01 = 0.110001 x 2^4

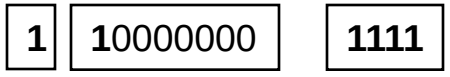
mantissa exponent



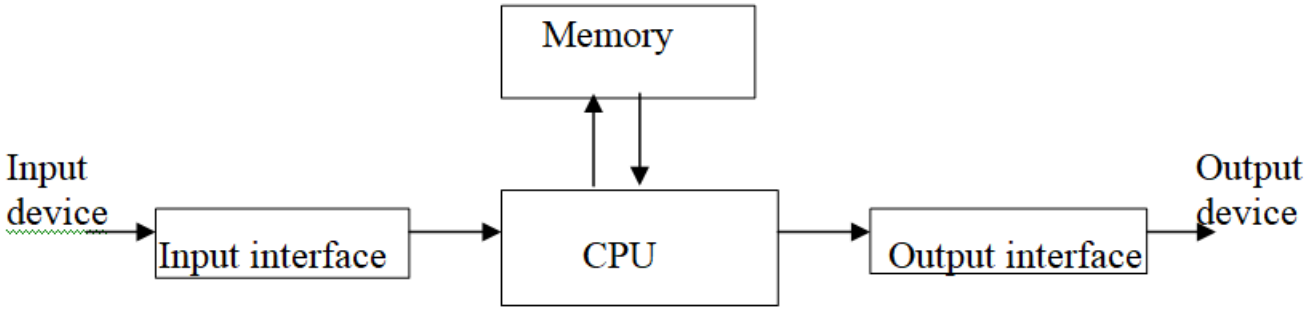
Ex: - 0.25

0.01 = 0.1 x 2^{-1}

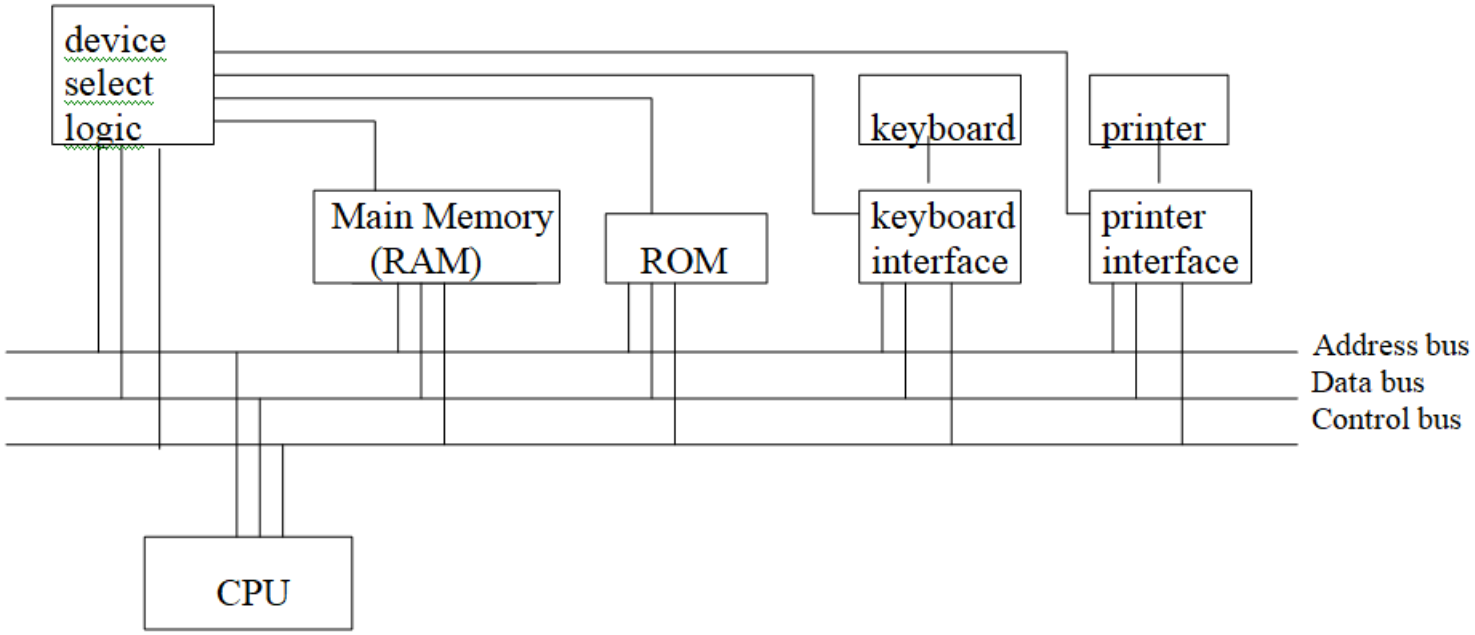
mantissa exponent



Basic Computer Hardware



Basic Computer Hardware



Basic Computer Hardware

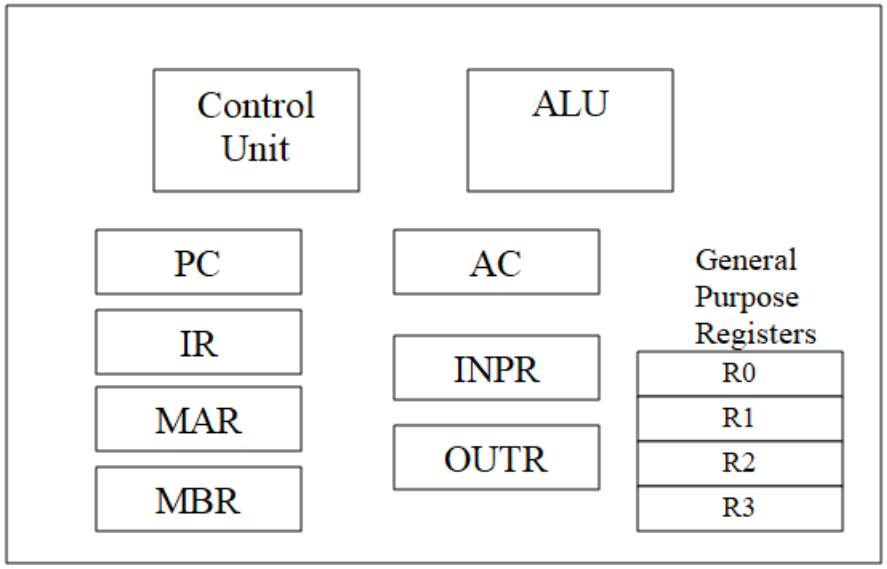
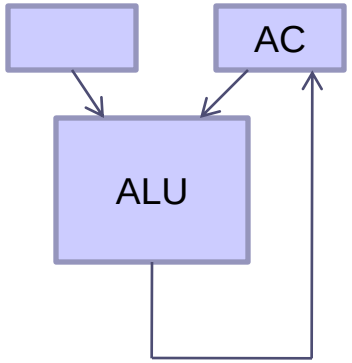


Figure 4. Inside the *CPU*



Basic Computer Hardware

Instruction Execution Cycle

1. Fetch instruction
2. Decode instruction
3. Fetch operands
4. Execute instruction
5. Store result

Basic Computer Hardware

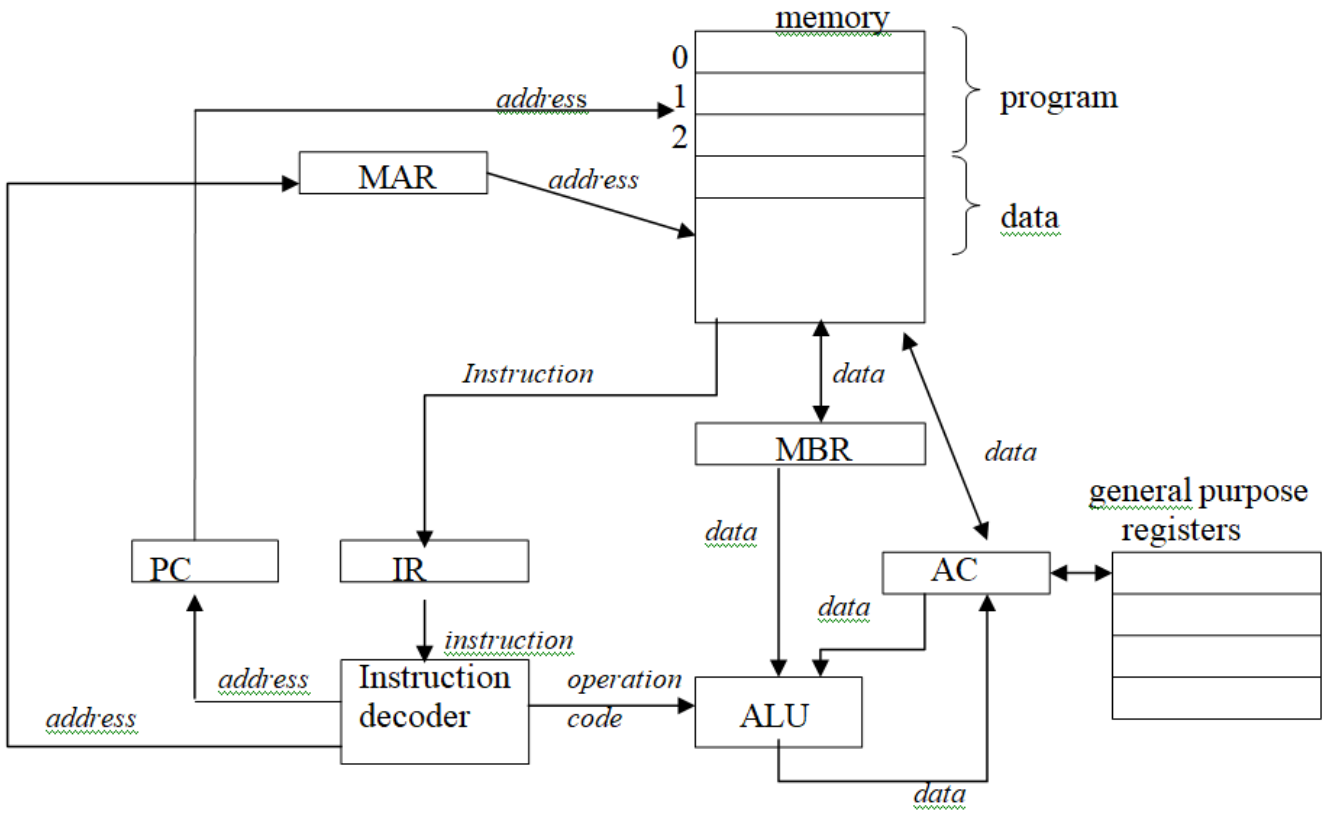


Figure 5. *Information flow between the components of a computer system*

Basic Computer Hardware

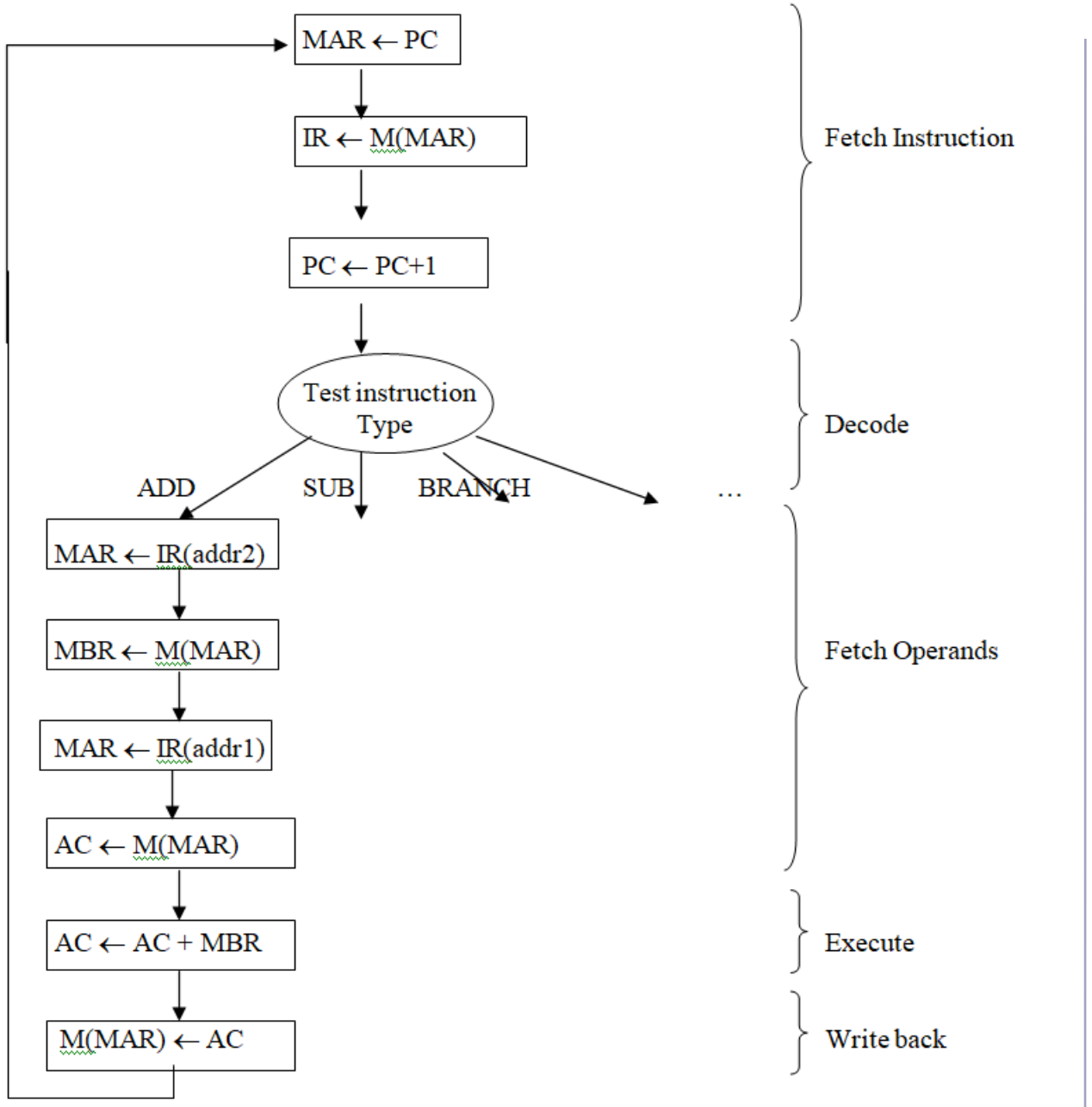
Machine instruction format

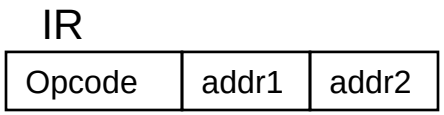
Ex:

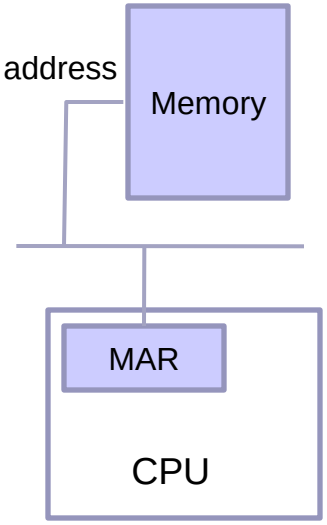
Opcode	Memory address	Memory address

ADD 100, 200

M[100] ← M[100] + M[200]


MAR ← PC
IR ← M(MAR)
PC ← PC+1
Test instruction Type
ADD
SUB
BRANCH
…
MAR ← IR(addr2)
MBR ← M(MAR)
MAR ← IR(addr1)
AC ← M(MAR)
AC ← AC + MBR
M(MAR) ← AC
Fetch Instruction
Decode
Fetch Operands
Execute
Write back


IR
Opcode
addr1
addr2


address
Memory
MAR
CPU

Instruction Formats

Information in an Instruction

1. Operation code (opcode)
2. Operand or Operand address
3. Addressing Mode

Instruction Formats

1. Operations

- Arithmetic and Logic
- Data transfer (load/store, move, etc.)
- Control (branch, jump, call/return)
- System
- Floating-point
- Special purpose operations (decimal, string, etc.)

Instruction Formats

2. Operands

• 3-operand format	ADD R1, R2, R3	R1 ← R2 + R3
• 2-operand format	ADD 100, 200	M[100] ← M[100] + M[200]
• 1-operand format	ADD 100	AC← AC + M[100]
• 0-operand format	CLRA	AC← 0

Instruction Formats

3. Addressing Modes

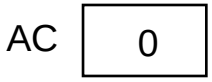
- Implied Addressing
- Immediate Addressing
- Direct Addressing
- Indirect Addressing
- Register Addressing
- Register Indirect Addressing
- Indexed Addressing
- Relative Addressing

Instruction Formats

Effective Address (EA): The actual memory address specified by an addressing mode

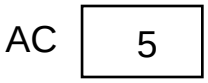
- Implied Addressing

CLRA AC← 0



- Immediate Addressing

LOAD #5 AC← 5

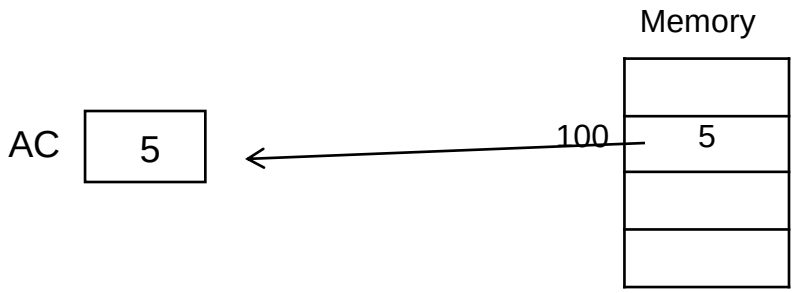


Instruction Formats

- Direct Addressing

LOAD 100 AC← M[100]

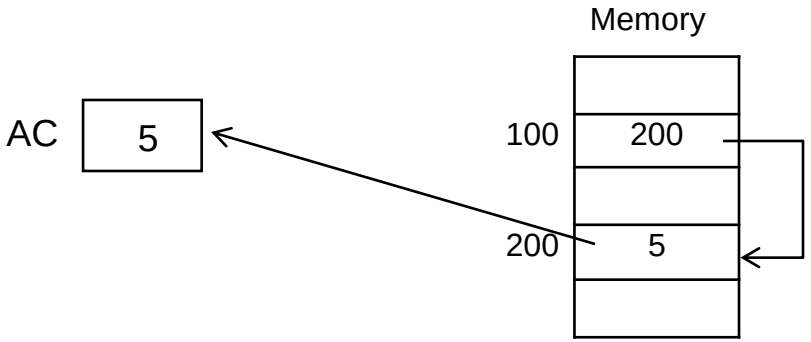
EA= 100



- Indirect Addressing

LOADI 100 AC← M[M[100]]

EA=200

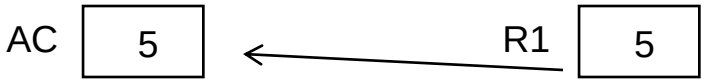


Instruction Formats

- Register Addressing

LOAD R1 AC← R1

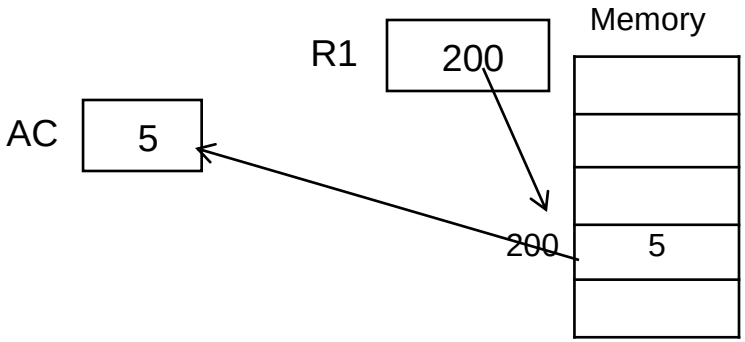
EA= R1



- Register Indirect Addressing

LOADI R1 AC← M[R1]

EA = 200



Instruction Formats

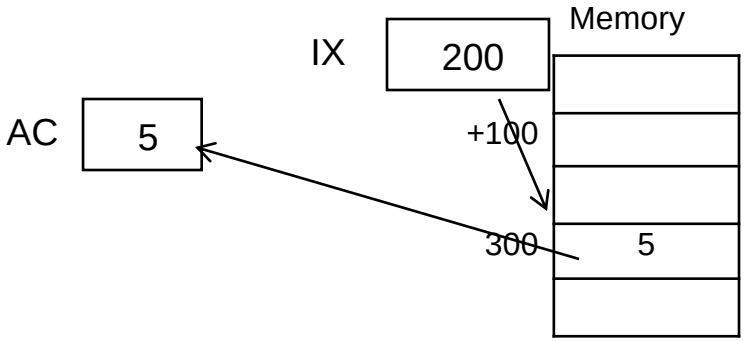
- Indexed Addressing

LOAD 100, X AC← M[100+IX]

offset

EA = IX + offset

= 200 + 100 = 300

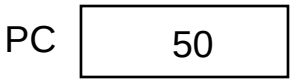


Instruction Formats

Direct Addressing

JUMP 50 PC← 50

EA = 50



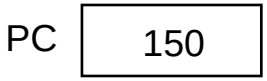
- Relative Addressing

BRANCH 50 PC← EA

offset

EA = PC + offset

= 100 + 50 = 150



Instruction Formats

Ex

Address	Memory
…	…
200	**LOAD 500**
…	…
400	700
…	…
500	800
…	…
600	900
…	…
800	300
…	…

IX | 100 |

Addressing Mode	EA	AC
Immediate	--	500
Direct	500	800
Indirect	800	300
Indexed	600	900

Instruction Formats

Ex

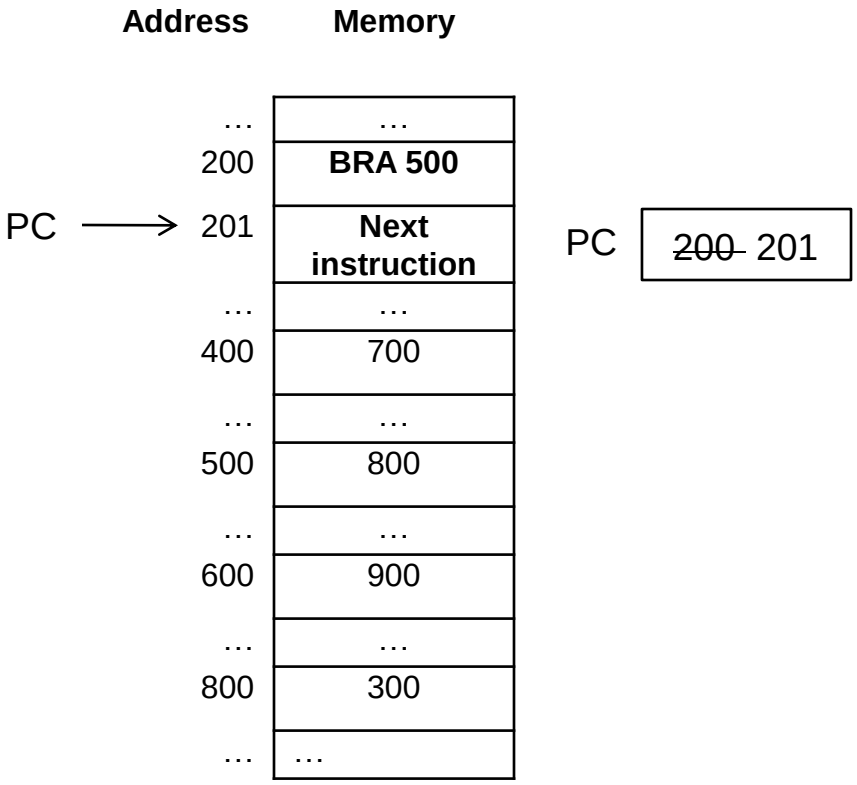
Address	Memory
…	…
200	**LOAD R1**
…	…
400	700
…	…
500	800
…	…
600	900
…	…
800	300
…	…

R1 | 400 |

Addressing Mode	EA	AC
Register	-	400
Register Indirect	400	700

Instruction Formats

Ex



PC ~~200~~ 201

Addressing Mode	EA	AC
Direct	500	--
Indirect	800	--
Relative	701	--

Systems Programming

System Programs (System Software): is a software designed to provide a platform for other software

